

CAMPUSCONECTA

Evaluación funcional y calidad de una plataforma web de interacción universitaria

Pruebas, experiencia de usuario, validación y resultados

Victoria Isabel Lavarello Vidaurre

Código 2023077281

Proyecto académico de ingeniería de software
4 de octubre de 2026

Resumen

Este artículo evalúa la calidad funcional de CampusConecta, una plataforma web para la interacción entre estudiantes, docentes y personal universitario. La evaluación tradujo los requisitos en criterios observables, combinó pruebas automáticas de integración, revisión del frontend, ejecución contra PostgreSQL real y verificación visual de los recorridos principales. Se aprobaron seis pruebas automáticas sin fallos ni omisiones; el frontend superó el análisis estático y la compilación; y el flujo de aceptación confirmó inicio de sesión, publicación, comentario, reacción, búsqueda y actualización del panel. La revisión visual documentó ocho estados del producto, incluidos autenticación, feed, comunidades, perfil y Swagger. El análisis se organiza con criterios de adecuación funcional, fiabilidad, usabilidad, seguridad y mantenibilidad inspirados en ISO/IEC 25010:2023. Los resultados respaldan el uso académico del sistema dentro del alcance probado, aunque todavía se requieren pruebas con usuarios, accesibilidad asistida, concurrencia y seguridad especializada antes de un despliegue institucional.

Palabras clave calidad de software, pruebas funcionales, experiencia de usuario, validación, ISO 25010, red social universitaria

Introducción

Una plataforma social resulta útil sólo si sus recorridos permiten completar las tareas previstas y conservan los cambios. Los perfiles, el contenido generado por usuarios y las interacciones forman parte de la estructura que Boyd y Ellison (2007) describen para los sitios de redes sociales. En CampusConecta, esa estructura se adaptó al contexto universitario mediante perfiles con rol y carrera o área, publicaciones, comentarios, reacciones y comunidades.

La evaluación se centró en resultados observables. No bastó con comprobar que la interfaz mostraba botones. Cada acción crítica debía enviar una solicitud válida, obtener una respuesta coherente, cambiar la base de datos y reflejar el resultado en otra vista. El estudio también examinó mensajes de error, protección de rutas y consistencia visual. ISO/IEC 25010:2023 se utilizó como marco para ordenar los hallazgos, sin afirmar una certificación formal.

Objetivo de evaluación

El objetivo fue determinar si CampusConecta satisface las funciones solicitadas y ofrece una experiencia coherente en un entorno local controlado. La evaluación debía identificar defectos funcionales, validar reglas esenciales y producir evidencia repetible. Se definieron criterios de aceptación para autenticación, perfiles, contenido, comunidades, búsqueda, panel, persistencia, manejo de errores y documentación de la API.

Marco de calidad

ISO/IEC 25010:2023 define un modelo de calidad de producto aplicable a sistemas de información. Para este trabajo se seleccionaron cinco características relacionadas con el alcance y las evidencias disponibles. La adecuación funcional examina si el producto resuelve las tareas; la fiabilidad observa respuestas estables ante casos previstos; la usabilidad considera claridad y continuidad de los recorridos; la seguridad revisa autenticación y permisos; y la mantenibilidad se aproxima mediante organización del código, pruebas y documentación.

Característica	Pregunta de evaluación	Evidencia
Adecuación funcional	¿Cada requisito tiene un flujo ejecutable?	Pruebas HTTP y capturas
Fiabilidad	¿La API responde de forma consistente y conserva datos?	6 pruebas y PostgreSQL real
Usabilidad	¿El usuario reconoce el estado y completa tareas?	Revisión de 8 pantallas
Seguridad	¿Las rutas y acciones respetan la identidad?	JWT, 401 y propiedad
Mantenibilidad	¿La solución puede verificarse y modificarse?	Capas, Swagger y scripts

Estrategia de pruebas

La estrategia combinó cuatro niveles. Las pruebas automáticas ejecutaron la API en memoria para aislar reglas y contratos. La compilación y el análisis estático detectaron errores de tipos y problemas del frontend. La prueba integral inició la API con PostgreSQL real y ejecutó un recorrido completo mediante HTTP. La revisión visual abrió la aplicación y registró ocho capturas de estados representativos. Esta combinación reduce el riesgo de aceptar una interfaz que no persiste o una API correcta que no puede usarse desde el navegador.

Nivel	Herramienta o técnica	Propósito
Integración API	xUnit y WebApplicationFactory	Validar contratos, reglas y autorización
Calidad frontend	Oxlint y compilación TypeScript/Vite	Detectar errores estáticos y producir build
Extremo a extremo	HTTP contra API y PostgreSQL real	Confirmar persistencia y lectura cruzada
Revisión visual	Navegador y capturas	Comprobar jerarquía, mensajes y adaptación
Exploratoria	20 consultas locales al feed	Obtener una línea base de latencia

Criterios de aceptación

- Una cuenta con datos válidos se registra y recibe un token; una contraseña débil se rechaza con 400.
- Una cuenta existente inicia sesión y puede consultar o editar su perfil.
- El feed recupera publicaciones persistidas y permite crear contenido autenticado.
- Los comentarios y las reacciones cambian la publicación y aparecen al volver a consultar.
- Las comunidades permiten crear, unirse y salir sin duplicar membresías.
- La búsqueda devuelve coincidencias de usuarios, publicaciones y grupos.
- El panel calcula indicadores del usuario a partir de datos persistidos.
- Un cliente sin JWT recibe 401 al acceder a una ruta protegida.
- Los errores se muestran sin exponer detalles internos del servidor.

Ejecución y resultados

La suite del backend ejecutó seis casos y terminó con seis aprobados, cero fallidos y cero omitidos. Una prueba comprobó el endpoint de salud. Otra verificó que la contraseña inicial estaba almacenada como hash y podía

validarse con PasswordHasher. El caso de registro rechazó una contraseña débil y aceptó una combinación válida. Los casos restantes recorrieron contenido, panel, comunidades, búsqueda, perfil y una ruta protegida sin token.

Caso	Resultado observado	Estado
Salud	GET /health respondió healthy	Aprobado
Contraseña	El hash inicial fue verificable sin almacenar texto plano	Aprobado
Registro	Contraseña débil rechazada; usuario válido recibió JWT	Aprobado
Contenido	Feed, post, comentario, reacción y panel funcionaron	Aprobado
Comunidad y perfil	Creación, búsqueda y edición respondieron correctamente	Aprobado
Autorización	Dashboard sin token devolvió 401	Aprobado

El frontend superó Oxlint sin advertencias y Vite produjo el paquete de producción. La compilación confirmó que los tipos y las importaciones eran válidos, aunque no reemplaza las pruebas de comportamiento. El flujo con PostgreSQL real añadió una publicación, un comentario y una reacción. La consulta posterior localizó el contenido y el panel mostró dos publicaciones propias, un comentario, dos reacciones recibidas y una comunidad.

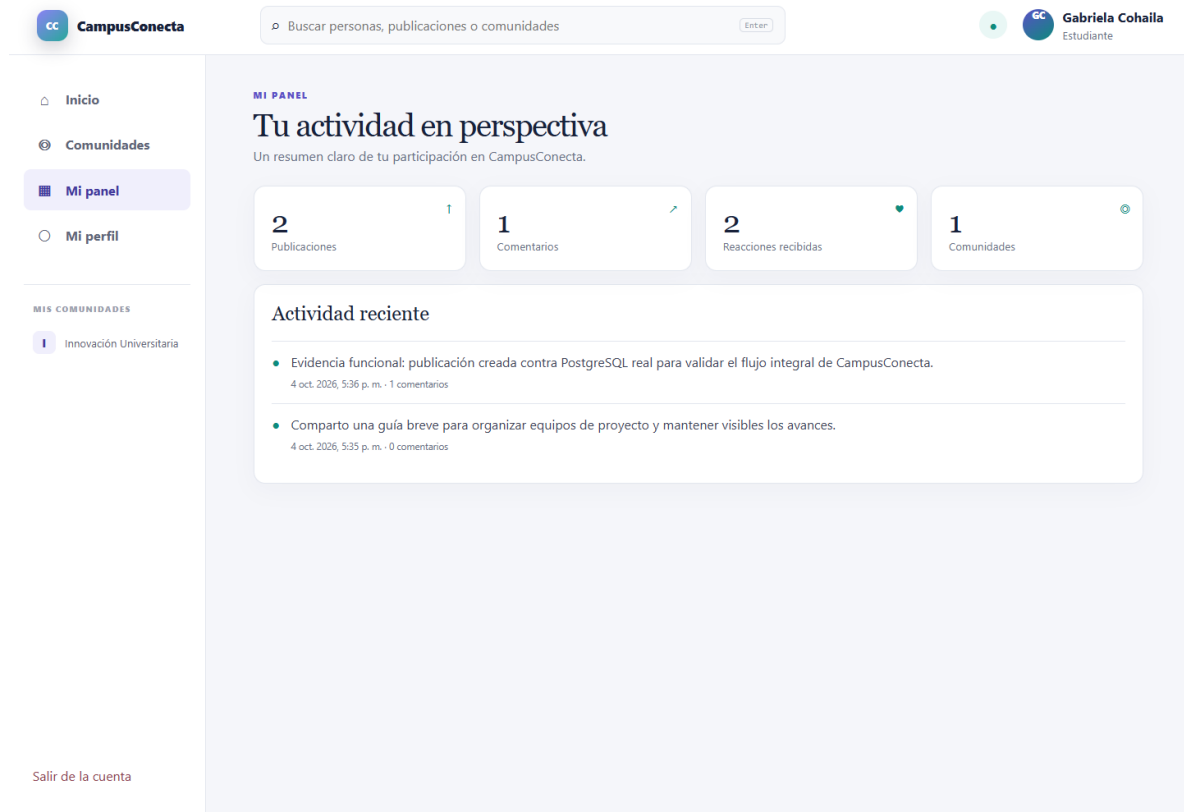


Figura 1 Panel personal después del flujo de aceptación

Evaluación de funcionalidades

La autenticación presentó dos formularios separados para ingreso y registro. Los estados de espera evitan envíos repetidos y la API mantiene la decisión final sobre la validez. Después del ingreso, la navegación muestra la identidad activa. El perfil permite modificar nombre visible, biografía, carrera o área y URL de avatar. La captura de edición confirma que los campos son identificables y que la acción principal se diferencia de la navegación.

El feed concentra la tarea más frecuente. Cada tarjeta identifica autor, fecha, comunidad cuando corresponde, contenido, reacciones y comentarios. La creación de una publicación actualiza la lista sin recargar todo el sitio. Los comentarios conservan su autoría y las reacciones usan un conjunto controlado de valores. La gestión básica permite eliminar contenido propio; la autorización también se comprueba en el servidor para impedir que la interfaz sea el único control.

Las comunidades muestran nombre, descripción, creador, número de miembros y estado de membresía. Crear una comunidad incorpora al creador como miembro. Unirse y salir actualiza el estado visible. La búsqueda agrupa usuarios, publicaciones y comunidades, de modo que una sola consulta puede conducir a distintos tipos de resultado. El panel resume la actividad personal y ofrece un punto de retorno al contenido reciente.

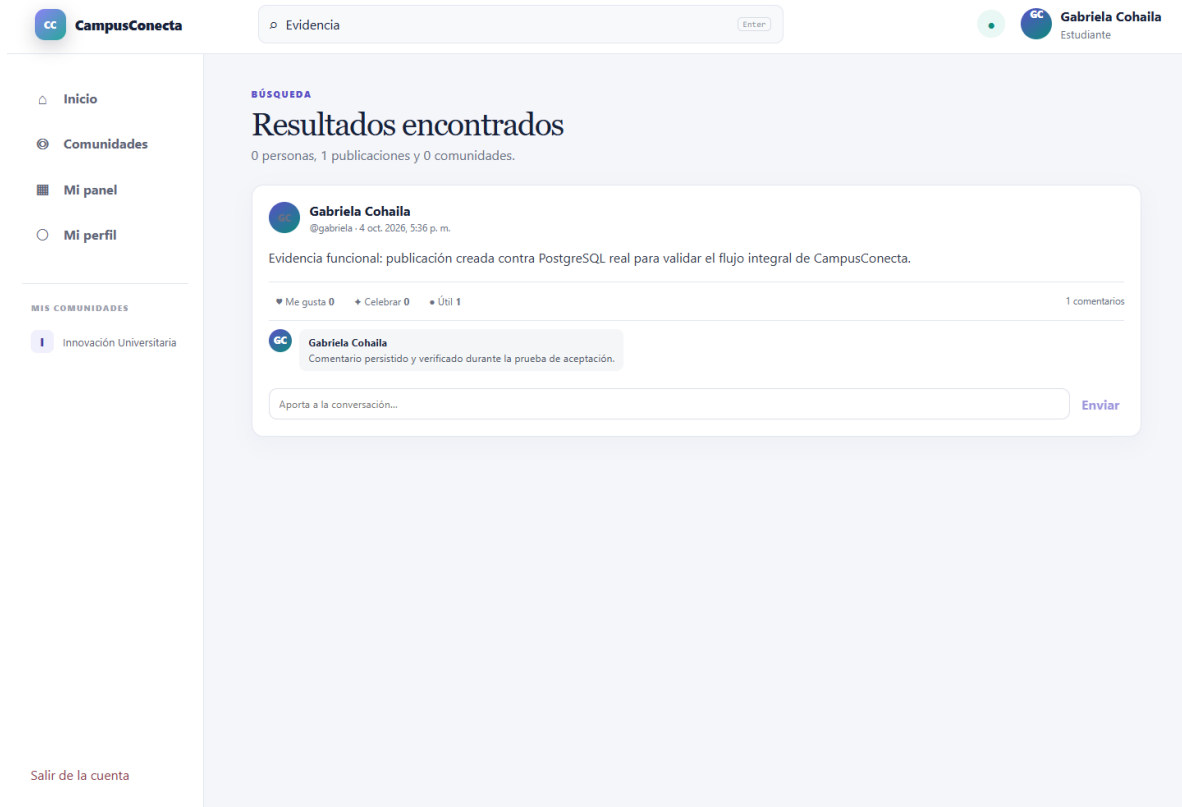


Figura 2 Búsqueda integrada con resultados persistentes

Experiencia de usuario

La revisión visual cubrió el acceso inicial, los formularios de autenticación, el feed, comunidades, panel, perfil, búsqueda y Swagger. Las pantallas mantuvieron la misma paleta, espaciado y jerarquía. El contenido principal se adaptó a una ventana estrecha durante la inspección manual y no presentó solapamientos. Los botones de acción muestran verbos concretos, y los mensajes de error proceden de una capa común de la API.

La aplicación usa etiquetas asociadas a los campos, orden visual estable y contraste claro entre texto y fondo. Estas observaciones son compatibles con principios generales de WCAG 2.2, pero no constituyen una conformidad certificada. No se ejecutó una auditoría completa con lector de pantalla, navegación exclusivamente por teclado ni medición automatizada de contraste. Por ello, la accesibilidad se considera parcialmente evaluada.

CC CampusConecta

Buscar personas, publicaciones o comunidades

GC Gabriela Cohaila Estudiante

Inicio

Comunidades

Mi panel

Mi perfil

MIS COMUNIDADES

Innovación Universitaria

Salir de la cuenta

MI PERFIL

GC Gabriela Cohaila @gabriela · Estudiante

Información personal

Nombre visible: Gabriela Cohaila

Facultad o área: Ingeniería

Biografía: Miembro de la comunidad universitaria en Ingeniería.

URL de fotografía:

Guardar perfil

Figura 3 Edición de perfil con campos y acción principal visibles

Validación y manejo de errores

La validación se divide entre cliente, contratos y reglas de dominio. El navegador evita solicitudes evidentemente incompletas. Los modelos de entrada especifican longitudes y formatos. Los controladores comprueban duplicados, recursos inexistentes, pertenencia y propiedad. El middleware devuelve una estructura de error en lugar de una página técnica. Esta defensa por capas conserva una experiencia clara sin confiar en datos enviados por el cliente.

Condición	Respuesta esperada	Efecto visible
Contraseña débil	400 Bad Request	El registro no se crea y se informa la regla
Credenciales incorrectas	401 Unauthorized	La sesión permanece cerrada
JWT ausente	401 Unauthorized	La ruta protegida no entrega datos
Recurso inexistente	404 Not Found	Se informa que no se encontró
Acción sobre contenido ajeno	403 Forbidden	No se modifica el recurso
Dato duplicado	400 Bad Request	No se crea una segunda identidad o membresía

Rendimiento exploratorio

Se enviaron veinte consultas consecutivas al feed desde el mismo equipo que alojaba la API. La media fue 5.40 ms, la mediana 4.73 ms, el percentil 95 9.58 ms, el mínimo 3.27 ms y el máximo 11.21 ms. La dispersión fue pequeña para esta muestra y no se observaron errores. El resultado aporta una referencia local para detectar cambios posteriores.

La medición no incluyó concurrencia, latencia de Internet, contenedores distribuidos ni volúmenes grandes. Tampoco mide el tiempo completo de pintura del navegador. Por estas razones, no se usa para afirmar cuántos usuarios soportaría el sistema. Una evaluación productiva debería fijar escenarios, datos representativos y umbrales antes de ejecutar pruebas de carga.

Hallazgos y mejoras

La evaluación encontró cobertura completa de las funciones mínimas y coherencia entre API, base e interfaz. Durante el desarrollo se corrigieron advertencias de consultas complejas mediante división explícita de consultas y se ordenaron resultados antes de limitar búsquedas. También se evitó atribuir al backend una demora originada por la resolución local de IPv6; las cifras finales se tomaron directamente sobre 127.0.0.1.

Prioridad	Mejora propuesta	Razón
Alta	Incorporar OpenID Connect institucional y secretos administrados	Reduce el riesgo operativo de identidad y claves
Alta	Ejecutar pruebas de autorización negativas por cada recurso	Amplía la cobertura más allá del dashboard
Media	Auditar WCAG 2.2 con teclado y lector de pantalla	Convierte observaciones visuales en evidencia accesible
Media	Añadir pruebas de carga y datos volumétricos	Permite estimar capacidad y cuellos de botella
Media	Agregar moderación con roles, reportes y auditoría	Escala la gestión básica del contenido
Baja	Incorporar notificaciones y recuperación de cuenta	Completa la experiencia de uso prolongado

Limitaciones

La evaluación fue realizada por el equipo desarrollador en un entorno local y con datos de demostración. No participaron usuarios externos ni se comparó el producto con otra plataforma. Las pruebas automáticas se concentraron en los recorridos críticos y no cubren todas las combinaciones de límites. La revisión visual documenta estados correctos, pero una captura no prueba por sí misma la accesibilidad ni la ausencia de defectos en todos los dispositivos.

Conclusiones

CampusConecta cumple los requisitos funcionales definidos para esta entrega. La evidencia conecta cada acción principal con una respuesta HTTP, un cambio persistido y una representación visible. Las seis pruebas aprobadas, la compilación del frontend y el flujo sobre PostgreSQL real reducen el riesgo de que el resultado sea sólo demostrativo.

La calidad observada es adecuada para una presentación académica y una evolución controlada. Antes de un uso institucional se deben completar pruebas con usuarios, accesibilidad, concurrencia y seguridad. El mayor valor del proceso fue convertir requisitos generales en criterios verificables y dejar scripts, documentación y evidencias que permiten repetir la evaluación.

Referencias

boyd, d. m., y Ellison, N. B. (2007). Social Network Sites: Definition, History, and Scholarship. *Journal of Computer-Mediated Communication*, 13(1), 210–230. <https://doi.org/10.1111/j.1083-6101.2007.00393.x>

- International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation, product quality model. <https://www.iso.org/standard/78176.html>
- World Wide Web Consortium. (2024). Web Content Accessibility Guidelines 2.2. <https://www.w3.org/TR/WCAG22/>
- Microsoft. (2026). ASP.NET Core web API documentation with Swagger and OpenAPI. <https://learn.microsoft.com/aspnet/core/tutorials/web-api-help-pages-using-swagger>
- OWASP Foundation. (2026). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- React Team. (2026). React documentation. <https://react.dev/learn>